

Redes Neuronales (2026)

Ayudas e indicaciones para la Guía de ejercicios N.º 4

(Trabajo en progreso)

Reportar errores a: `tristan.osan@unc.edu.ar`**Índice**

Integración numérica de ecuaciones diferenciales ordinarias (EDOs)	2
1. Ejercicio 1: Importando librerías	3
2. Ejercicio 2: Implementando el Método de Euler	3
2.1. La función <code>euler</code>	3
2.2. La función <code>integrador_ode</code>	4
2.3. Resultado esperado	6
3. Ejercicio 6: Oscilador armónico	7
3.1. Reducción del orden de una EDO	7
3.2. El oscilador armónico y el sistema que se integra	7
3.3. Compatibilidad con <code>integrador_ode</code> : número de pasos	8
3.4. Fórmula iterativa de Euler para este problema	8
3.5. Primeros 10 pasos iterativos: cálculo algebraico	8
3.6. Tabla de la condición inicial y los primeros 10 pasos	10
3.7. Implementación en Python usando el integrador del Ejercicio 2	10
3.8. Resultado gráfico esperado	11
3.9. Chequeo analítico y una observación sobre Euler	12
4. Ejercicio 7: Integrador de RK4	12
4.1. Implementación del método de RK4	12
4.2. Oscilador armónico integrado con RK4	13
4.3. Comparación entre Euler y RK4	17

Integración numérica de ecuaciones diferenciales ordinarias (EDOs)

El método de Euler

El método de Euler es un procedimiento numérico de primer orden utilizado para resolver problemas de valor inicial (PVI) de la forma

$$\boxed{\begin{aligned} \frac{dx}{dt} &= f(x(t), t, \mathbf{p}), \\ x(t_0) &= x_0 \quad (\text{Condición inicial}). \end{aligned}} \quad (1)$$

donde $\mathbf{p}$ representa el conjunto de parámetros del modelo. A pesar de su simplicidad, el método de Euler es un paso importante hacia técnicas más sofisticadas en análisis numérico.

Derivación del método de Euler

El método de Euler se deriva de la expansión en serie de Taylor de la solución $x(t)$ alrededor de un punto t_n :

$$x(t_{n+1}) = x(t_n) + \Delta t \left. \frac{dx}{dt} \right|_{t=t_n} + \frac{\Delta t^2}{2} \left. \frac{d^2x}{dt^2} \right|_{t=t_n} + \mathcal{O}(\Delta t^3) \quad (2)$$

A primer orden en Δt tenemos,

$$x(t_{n+1}) \approx x(t_n) + \Delta t f(x(t_n), t_n, \mathbf{p}), \quad (3)$$

lo cual conduce a la fórmula de actualización

$$\boxed{x_{n+1} = x_n + \Delta t f(x_n, t_n, \mathbf{p})}, \quad (4)$$

donde $x_n \doteq x(t_n)$.

Ejemplo: crecimiento exponencial

Consideremos

$$\frac{dx}{dt} = kx, \quad x(0) = x_0, \quad (5)$$

donde k es una constante. La solución exacta es $x(t) = x_0 e^{kt}$, sin embargo, vamos a aplicar el método de Euler. De la ecuación (5) identificamos $f(x, t, \mathbf{p}) = kx$ (con $\mathbf{p} = (k)$) y, aplicando Euler,

$$x_{n+1} = x_n + \Delta t k x_n \quad (6)$$

$$= x_n (1 + k \Delta t). \quad (7)$$

Es decir, comenzando desde x_0 , cada valor subsecuente x_n ($n = 1, 2, 3, \dots$) se calcula iterativamente.

1. Ejercicio 1: Importando librerías

```
import numpy as np
import scipy as sp
import scipy.linalg as linalg
import matplotlib.pyplot as plt
```

2. Ejercicio 2: Implementando el Método de Euler

La versatilidad de Python nos permite encapsular la lógica del método de Euler en una función reutilizable. Esto hace que sea más fácil aplicar el método a cualquier ecuación diferencial, simplemente cambiando la función que describe el lado derecho de la ecuación.

2.1. La función euler

En la implementación escribiremos $h = \Delta t$. La función `euler` se implementa directamente a partir de

$$\mathbf{x}(t+h) \approx \mathbf{x}(t) + h \mathbf{f}(\mathbf{x}(t), t, \mathbf{p}). \quad (8)$$

```
import numpy as np

def euler(f, x, t, h, p):
    """
    Calcula un paso de integración de una EDO usando el método de Euler.

    Argumentos:
        f : función (x, t, p) -> array, que define el lado derecho de la EDO.
        x : array, el estado actual.
        t : float, el tiempo actual.
        h : float, el paso de integración.
        p : array, un vector de parámetros.

    Retorna:
        array, el estado en el siguiente paso de tiempo, x(t+h).
    """
    x_siguiente = x + h * f(x, t, p)
    return x_siguiente

# --- EJEMPLO DE USO ---
# EDO de decaimiento exponencial: dx/dt = -k*x

def decaimiento_exponencial(x, t, p):
    k = p[0]
    return -k * x

x0 = np.array([100.0])
t0 = 0.0
h_step = 0.1
parametros = np.array([0.5])

x1 = euler(decaimiento_exponencial, x0, t0, h_step, parametros)
print(f"Estado inicial x(t=0): {x0}")
print(f"Estado siguiente x(t={t0+h_step}): {x1}")
```

```
# Resultado esperado:
# Estado inicial x(t=0): [100.]
# Estado siguiente x(t=0.1): [95.]
```

Listing 1: Función para un paso del método de Euler

La función `decaimiento_exponencial` caracteriza el lado derecho de la EDO. El estado inicial se representa mediante un array para mantener la notación vectorial, y el vector de parámetros $\mathbf{p}$ contiene la constante k . En este caso,

$$x(0.1) = x(0) + 0.1(-0.5x(0)) = 100 + 0.1(-50) = 95.$$

2.2. La función `integrador_ode`

La función `integrador_ode` automatiza el proceso de llamar repetidamente a `euler` para calcular la trayectoria de la solución a lo largo de un intervalo de tiempo.

```
import numpy as np
import matplotlib.pyplot as plt

def euler(f, x, t, h, p):
    return x + h * f(x, t, p)

def integrador_ode(m, f, x0, t0, tf, npasos, p):
    """
    Integra una EDO desde un tiempo inicial a uno final usando un método dado.

    Argumentos:
        m      : str, el método a usar ('e' para Euler).
        f      : función (x, t, p), lado derecho de la EDO.
        x0     : array, condición inicial x(t0).
        t0     : float, tiempo inicial.
        tf     : float, tiempo final.
        npasos : int, número de pasos de integración.
        p      : array, vector de parámetros.

    Retorna:
        (array, array), vector de tiempos y matriz de estados.
    """
    h = (tf - t0) / npasos

    t_puntos = np.zeros(npasos + 1)
    x_puntos = np.zeros((npasos + 1, len(x0)))

    t_puntos[0] = t0
    x_puntos[0] = x0

    if m == 'e':
        for i in range(npasos):
            x_actual = x_puntos[i]
            t_actual = t_puntos[i]

            x_siguiente = euler(f, x_actual, t_actual, h, p)

            x_puntos[i+1] = x_siguiente
            t_puntos[i+1] = t_actual + h
    else:
```

```

        raise ValueError("Método no soportado. Use 'e' para Euler.")

    return t_puntos, x_puntos

# --- EJEMPLO DE USO ---

def decaimiento_exponencial(x, t, p):
    k = p[0]
    return -k * x

x_inicial = np.array([100.0])
tiempo_inicial = 0.0
tiempo_final = 10.0
n_pasos = 50
parametros_k = np.array([0.5])

tiempos, solucion = integrador_ode(
    'e',
    decaimiento_exponencial,
    x_inicial,
    tiempo_inicial,
    tiempo_final,
    n_pasos,
    parametros_k
)

# --- TABLA DE RESULTADOS ---

print("Tiempo (t) | Solución (x)")
print("-----")

for i in range(0, n_pasos + 1):
    print(f"{tiempos[i]:.2f} | {solucion[i][0]:.4f}")

# --- GRÁFICO ---

# Solución exacta
k = parametros_k[0]
x_exacta = x_inicial[0] * np.exp(-k * tiempos)

plt.figure(figsize=(8, 5))

# Solución numérica de Euler: círculos huecos
plt.plot(
    tiempos,
    solucion[:, 0],
    'o',
    markerfacecolor='none',
    label='Euler'
)

# Solución exacta: línea continua
plt.plot(
    tiempos,
    x_exacta,
    '-',
    label='Solución exacta'
)

```

```
plt.xlabel(r'$t$')
plt.ylabel(r'$x(t)$')
plt.title('Decaimiento exponencial')
plt.grid(True)
plt.legend()

plt.show()
```

Listing 2: Integrador numérico genérico para EDOs

La lógica del integrador puede resumirse así:

1. calcula $h = (t_f - t_0)/\text{npasos}$;
2. inicializa los arrays de tiempos y estados;
3. guarda la condición inicial;
4. itera **npasos** veces, tomando el estado actual, aplicando un paso de Euler y almacenando el nuevo estado;
5. retorna la historia completa de tiempos y estados.
6. Muestra un gráfico comparativo de la solución obtenida con el método de Euler y la solución exacta, para el decaimiento exponencial.

2.3. Resultado esperado

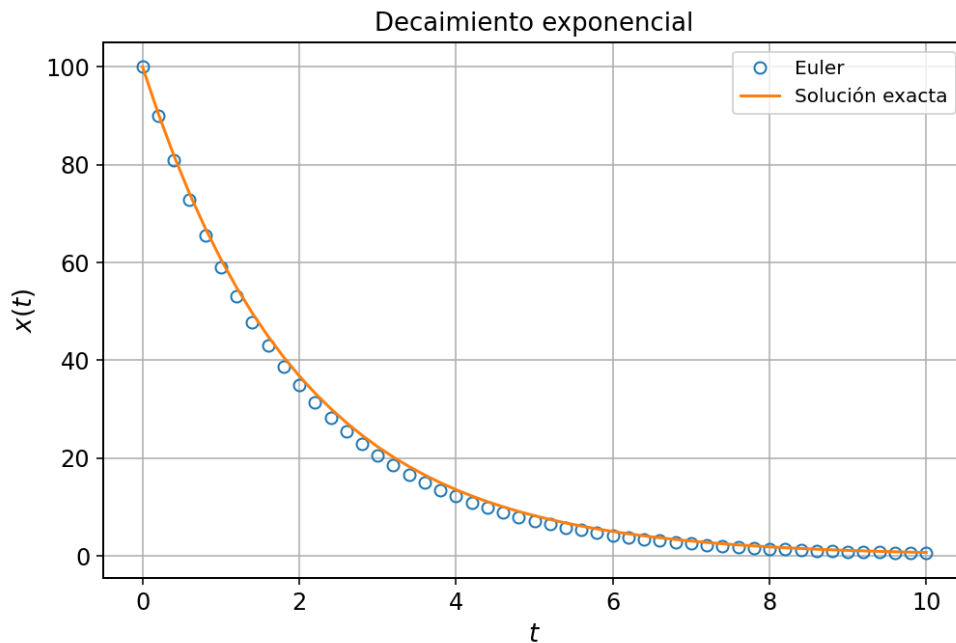


Figura 1: Comparación entre la solución numérica obtenida mediante el método de Euler y la solución exacta para el decaimiento exponencial.

3. Ejercicio 6: Oscilador armónico

3.1. Reducción del orden de una EDO

Comúnmente, una ecuación diferencial ordinaria (EDO) de orden n tiene la forma

$$y^{(n)}(t) = f(y(t), y'(t), y''(t), \dots, y^{(n-1)}(t), t). \quad (9)$$

donde $y^{(i)}$ denota la i -ésima derivada de y respecto de t . Si $y(t) \in \mathbb{R}^d$, esta ecuación puede reducirse a una EDO de primer orden y dimensión nd mediante

$$x_1(t) := y(t), \quad (10)$$

$$x_2(t) := y'(t) = x_1'(t), \quad (11)$$

$$x_3(t) := y''(t) = x_2'(t), \quad (12)$$

$$\vdots \quad (13)$$

$$x_n(t) := y^{(n-1)}(t) = x_{n-1}'(t). \quad (14)$$

Entonces,

$$x_1'(t) = x_2(t), \quad (15)$$

$$x_2'(t) = x_3(t), \quad (16)$$

$$\vdots \quad (17)$$

$$x_{n-1}'(t) = x_n(t), \quad (18)$$

$$x_n'(t) = f(x_1(t), x_2(t), \dots, x_n(t), t). \quad (19)$$

3.2. El oscilador armónico y el sistema que se integra

El oscilador armónico del ejercicio está descrito por

$$y''(t) = -\alpha y(t). \quad (20)$$

Esta ecuación es una EDO *escalar* de segundo orden. Definimos

$$x_1(t) = y(t), \quad x_2(t) = y'(t), \quad (21)$$

y obtenemos el sistema de primer orden

$$\boxed{\begin{aligned} x_1'(t) &= x_2(t), \\ x_2'(t) &= -\alpha x_1(t). \end{aligned}} \quad (22)$$

Para conservar exactamente la interfaz usada en este tutorial, introducimos el estado vectorial y el vector de parámetros

$$\mathbf{x}(t) = \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix}, \quad \mathbf{p} = (\alpha), \quad (23)$$

de modo que la función del lado derecho es

$$\boxed{\mathbf{f}(\mathbf{x}, t, \mathbf{p}) = \begin{pmatrix} x_2 \\ -\alpha x_1 \end{pmatrix}.} \quad (24)$$

Por lo tanto, este es el sistema de ecuaciones que debe recibir el integrador genérico del Ejercicio 2.

Las condiciones del ejercicio son

$$t_0 = a = 0, \quad t_f = b = 10, \quad x_1(0) = 1, \quad x_2(0) = 0, \quad \alpha = 3, \quad (25)$$

con paso deseado $h = 0.1$.

3.3. Compatibilidad con `integrador_ode`: número de pasos

En el tutorial, `integrador_ode` no recibe directamente h : recibe `npasos` y calcula

$$h = \frac{t_f - t_0}{\text{npasos}}.$$

Por lo tanto, para obtener $h = 0.1$ en $[0, 10]$ se debe usar

$$\boxed{\text{npasos} = \frac{10 - 0}{0.1} = 100.} \quad (26)$$

3.4. Fórmula iterativa de Euler para este problema

Aplicando la ecuación (8) al sistema (22),

$$x_{1,n+1} = x_{1,n} + h x_{2,n}, \quad (27)$$

$$x_{2,n+1} = x_{2,n} - h \alpha x_{1,n}. \quad (28)$$

Con $h = 0.1$ y $\alpha = 3$,

$$\boxed{\begin{aligned} x_{1,n+1} &= x_{1,n} + 0.1 x_{2,n}, \\ x_{2,n+1} &= x_{2,n} - 0.3 x_{1,n}. \end{aligned}} \quad (29)$$

Equivalentemente,

$$\begin{pmatrix} x_{1,n+1} \\ x_{2,n+1} \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0.1 \\ -0.3 & 1 \end{pmatrix}}_{A_E} \begin{pmatrix} x_{1,n} \\ x_{2,n} \end{pmatrix}, \quad (30)$$

donde A_E denota la *matriz de actualización*.

3.5. Primeros 10 pasos iterativos: cálculo algebraico

La condición inicial corresponde a $n = 0$:

$$(t_0, x_{1,0}, x_{2,0}) = (0, 1, 0).$$

A continuación se muestran las diez actualizaciones de Euler, es decir, desde $n = 0 \rightarrow 1$ hasta $n = 9 \rightarrow 10$.

Paso 1: $t_0 = 0 \rightarrow t_1 = 0.1$.

$$\begin{aligned} x_{1,1} &= 1 + 0.1(0) = 1, \\ x_{2,1} &= 0 - 0.3(1) = -0.3. \end{aligned}$$

Paso 2: $t_1 = 0.1 \rightarrow t_2 = 0.2$.

$$\begin{aligned}x_{1,2} &= 1 + 0.1(-0.3) = 0.97, \\x_{2,2} &= -0.3 - 0.3(1) = -0.60.\end{aligned}$$

Paso 3: $t_2 = 0.2 \rightarrow t_3 = 0.3$.

$$\begin{aligned}x_{1,3} &= 0.97 + 0.1(-0.60) = 0.910, \\x_{2,3} &= -0.60 - 0.3(0.97) = -0.891.\end{aligned}$$

Paso 4: $t_3 = 0.3 \rightarrow t_4 = 0.4$.

$$\begin{aligned}x_{1,4} &= 0.910 + 0.1(-0.891) = 0.8209, \\x_{2,4} &= -0.891 - 0.3(0.910) = -1.1640.\end{aligned}$$

Paso 5: $t_4 = 0.4 \rightarrow t_5 = 0.5$.

$$\begin{aligned}x_{1,5} &= 0.8209 + 0.1(-1.1640) = 0.70450, \\x_{2,5} &= -1.1640 - 0.3(0.8209) = -1.41027.\end{aligned}$$

Paso 6: $t_5 = 0.5 \rightarrow t_6 = 0.6$.

$$\begin{aligned}x_{1,6} &= 0.70450 + 0.1(-1.41027) = 0.563473, \\x_{2,6} &= -1.41027 - 0.3(0.70450) = -1.621620.\end{aligned}$$

Paso 7: $t_6 = 0.6 \rightarrow t_7 = 0.7$.

$$\begin{aligned}x_{1,7} &= 0.563473 + 0.1(-1.621620) = 0.4013110, \\x_{2,7} &= -1.621620 - 0.3(0.563473) = -1.7906619.\end{aligned}$$

Paso 8: $t_7 = 0.7 \rightarrow t_8 = 0.8$.

$$\begin{aligned}x_{1,8} &= 0.4013110 + 0.1(-1.7906619) = 0.22224481, \\x_{2,8} &= -1.7906619 - 0.3(0.4013110) = -1.91105520.\end{aligned}$$

Paso 9: $t_8 = 0.8 \rightarrow t_9 = 0.9$.

$$\begin{aligned}x_{1,9} &= 0.22224481 + 0.1(-1.91105520) = 0.031139290, \\x_{2,9} &= -1.91105520 - 0.3(0.22224481) = -1.977728643.\end{aligned}$$

Paso 10: $t_9 = 0.9 \rightarrow t_{10} = 1.0$.

$$\begin{aligned}x_{1,10} &= 0.031139290 + 0.1(-1.977728643) = -0.1666335743, \\x_{2,10} &= -1.977728643 - 0.3(0.031139290) = -1.9870704300.\end{aligned}$$

3.6. Tabla de la condición inicial y los primeros 10 pasos

La fila $n = 0$ es la condición inicial; las filas $n = 1, \dots, 10$ son las diez iteraciones solicitadas.

Cuadro 1: Primeras diez iteraciones de Euler para $\alpha = 3$ y $h = 0.1$.

n	t_n	$x_{1,n}$	$x_{2,n}$
0	0.0	1.0000000000	0.0000000000
1	0.1	1.0000000000	-0.3000000000
2	0.2	0.9700000000	-0.6000000000
3	0.3	0.9100000000	-0.8910000000
4	0.4	0.8209000000	-1.1640000000
5	0.5	0.7045000000	-1.4102700000
6	0.6	0.5634730000	-1.6216200000
7	0.7	0.4013110000	-1.7906619000
8	0.8	0.2222448100	-1.9110552000
9	0.9	0.0311392900	-1.9777286430
10	1.0	-0.1666335743	-1.9870704300

3.7. Implementación en Python usando el integrador del Ejercicio 2

La función que representa el lado derecho de la EDO debe respetar la firma $f(x, t, p)$ del tutorial:

```
import numpy as np
import matplotlib.pyplot as plt

# Se asume que euler(...) e integrador_ode(...) ya fueron definidos
# como en el Ejercicio 2.

def oscilador_armonico(x, t, p):
    alpha = p[0]
    x1 = x[0]
    x2 = x[1]
    return np.array([x2, -alpha * x1])

# Condiciones y parámetros del Ejercicio 6
x_inicial = np.array([1.0, 0.0])
tiempo_inicial = 0.0
tiempo_final = 10.0
h_step = 0.1
n_pasos = int((tiempo_final - tiempo_inicial) / h_step) # 100
parametros_alpha = np.array([3.0])

# Integración
tiempos, solucion = integrador_ode(
    'e', oscilador_armonico, x_inicial,
    tiempo_inicial, tiempo_final, n_pasos, parametros_alpha
)

x1 = solucion[:, 0]
x2 = solucion[:, 1]

# Primeros 10 pasos (más la condición inicial)
print(" n | t_n | x1_n | x2_n")
print("-----")
```

```

for n in range(11):
    print(f"{n:2d} | {tiempos[n]:.1f} | {x1[n]: .10f} | {x2[n]: .10f}")

# x1(t) y x2(t) vs t, usando puntos
plt.figure(figsize=(9, 5))
plt.scatter(tiempos, x1, s=14, label=r'$x_1(t)$')
plt.scatter(tiempos, x2, s=14, label=r'$x_2(t)$')
plt.xlabel('Tiempo t')
plt.ylabel('Estado')
plt.legend()
plt.grid(True)
plt.show()

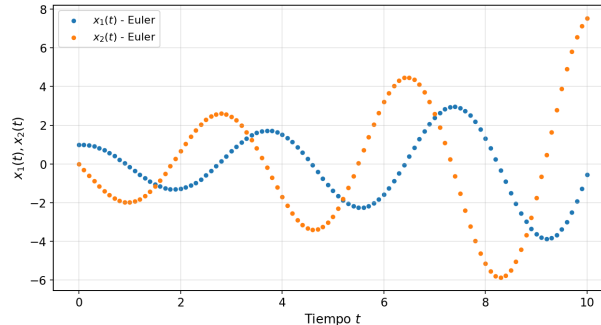
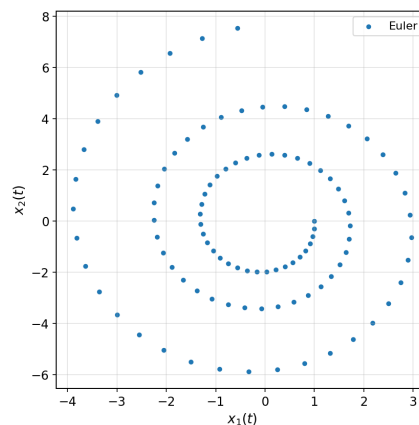
# Diagrama de fase: x2(t) frente a x1(t)
plt.figure(figsize=(6, 6))
plt.scatter(x1, x2, s=14, label=r'$x_1(t), x_2(t)$')
plt.xlabel(r'$x_1(t)$')
plt.ylabel(r'$x_2(t)$')
plt.legend()
plt.grid(True)
plt.show()

```

Listing 3: Oscilador armónico integrado con el método de Euler

3.8. Resultado gráfico esperado

Las siguientes figuras reproducen los puntos que genera el método de Euler explícito con $h = 0.1$ y $\alpha = 3$.

Figura 2: $x_1(t)$ y $x_2(t)$ obtenidos por el método de Euler.Figura 3: Diagrama de fases numérico con el método de Euler: $x_2(t)$ frente a $x_1(t)$.

3.9. Chequeo analítico y una observación sobre Euler

La solución exacta del problema es

$$x_1(t) = \cos(\sqrt{3}t), \quad x_2(t) = -\sqrt{3} \sin(\sqrt{3}t). \quad (31)$$

Esta solución exacta tiene amplitud acotada. Sin embargo, Euler explícito no conserva la energía del oscilador. En efecto, la matriz de actualización A_E tiene determinante

$$\det(A_E) = 1 + \alpha h^2 = 1 + 3(0.1)^2 = 1.03 > 1,$$

por lo que el diagrama de fases numérico se abre lentamente hacia afuera. Este comportamiento no es un error de implementación: es una propiedad del método de Euler explícito aplicado a este sistema oscilatorio.

4. Ejercicio 7: Integrador de RK4

El método de Runge–Kutta de orden 4 (RK4) mejora sustancialmente la aproximación de un paso de integración sin requerir derivadas explícitas de la función $\mathbf{f}$. Para mantener la notación utilizada en los Ejercicios 2 y 6, consideramos el problema vectorial

$$\mathbf{x}'(t) = \mathbf{f}(\mathbf{x}(t), t, \mathbf{p}). \quad (32)$$

A partir del estado numérico $\mathbf{x}_n$ en el tiempo t_n , RK4 define cuatro evaluaciones intermedias:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n, t_n, \mathbf{p}), \quad (33)$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{x}_n + \frac{h}{2}\mathbf{k}_1, t_n + \frac{h}{2}, \mathbf{p}\right), \quad (34)$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{x}_n + \frac{h}{2}\mathbf{k}_2, t_n + \frac{h}{2}, \mathbf{p}\right), \quad (35)$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_n + h\mathbf{k}_3, t_n + h, \mathbf{p}), \quad (36)$$

y calcula el siguiente estado mediante

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (37)$$

El error local de truncamiento de un paso es $\mathcal{O}(h^5)$; al integrar sobre un intervalo fijo, RK4 es un método de orden global 4, es decir, su error global decrece como $\mathcal{O}(h^4)$.

4.1. Implementación del método de RK4

La implementación sigue la misma interfaz que la función `euler`: recibe `f`, el estado actual `x`, el tiempo `t`, el paso `h` y el vector de parámetros `p`.

```
def rk4(f, x, t, h, p):
    """Computa un paso de integración con Runge-Kutta de orden 4."""
    k1 = f(x, t, p)
    k2 = f(x + 0.5 * h * k1, t + 0.5 * h, p)
    k3 = f(x + 0.5 * h * k2, t + 0.5 * h, p)
    k4 = f(x + h * k3, t + h, p)

    x_siguiente = x + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
    return x_siguiente
```

Listing 4: Función para un paso del método RK4

Extendemos ahora la función `integrador_ode` del Ejercicio 2 para poder seleccionar Euler o RK4. La única diferencia conceptual está en la rutina que calcula `x_siguiente`; el almacenamiento de tiempos y estados no cambia.

```
def integrador_ode(m, f, x0, t0, tf, npasos, p):
    h = (tf - t0) / npasos

    t_puntos = np.zeros(npasos + 1)
    x_puntos = np.zeros((npasos + 1, len(x0)))
    t_puntos[0] = t0
    x_puntos[0] = x0

    for i in range(npasos):
        x_actual = x_puntos[i]
        t_actual = t_puntos[i]

        if m == 'e':
            x_siguiente = euler(f, x_actual, t_actual, h, p)
        elif m == 'rk4':
            x_siguiente = rk4(f, x_actual, t_actual, h, p)
        else:
            raise ValueError("Método no soportado. Use 'e' o 'rk4'.")

        x_puntos[i+1] = x_siguiente
        t_puntos[i+1] = t_actual + h

    return t_puntos, x_puntos
```

Listing 5: Integrador genérico con Euler y RK4

4.2. Oscilador armónico integrado con RK4

Repetimos exactamente el problema del Ejercicio 6:

$$\boxed{\begin{matrix} x_1'(t) = x_2(t), \\ x_2'(t) = -\alpha x_1(t), \end{matrix}} \quad \mathbf{x}(0) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \alpha = 3, \quad (38)$$

con $t \in [0, 10]$ y $h = 0.1$. Como antes,

$$\text{npasos} = \frac{10 - 0}{0.1} = 100. \quad (39)$$

La función del lado derecho sigue siendo

$$\mathbf{f}(\mathbf{x}, t, \mathbf{p}) = \begin{pmatrix} x_2 \\ -\alpha x_1 \end{pmatrix}, \quad \mathbf{p} = (\alpha). \quad (40)$$

4.2.1. Primer paso de RK4, desarrollado algebraicamente

Partimos de

$$\mathbf{x}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad t_0 = 0, \quad h = 0.1, \quad \alpha = 3.$$

Entonces,

$$\begin{aligned}
 \mathbf{k}_1 &= \mathbf{f}(\mathbf{x}_0, t_0, \mathbf{p}) = \begin{pmatrix} 0 \\ -3 \end{pmatrix}, \\
 \mathbf{k}_2 &= \mathbf{f}\left(\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \frac{0.1}{2} \begin{pmatrix} 0 \\ -3 \end{pmatrix}, 0.05, \mathbf{p}\right) = \mathbf{f}\left(\begin{pmatrix} 1 \\ -0.15 \end{pmatrix}, 0.05, \mathbf{p}\right) = \begin{pmatrix} -0.15 \\ -3 \end{pmatrix}, \\
 \mathbf{k}_3 &= \mathbf{f}\left(\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \frac{0.1}{2} \begin{pmatrix} -0.15 \\ -3 \end{pmatrix}, 0.05, \mathbf{p}\right) = \mathbf{f}\left(\begin{pmatrix} 0.9925 \\ -0.15 \end{pmatrix}, 0.05, \mathbf{p}\right) = \begin{pmatrix} -0.15 \\ -2.9775 \end{pmatrix}, \\
 \mathbf{k}_4 &= \mathbf{f}\left(\begin{pmatrix} 1 \\ 0 \end{pmatrix} + 0.1 \begin{pmatrix} -0.15 \\ -2.9775 \end{pmatrix}, 0.1, \mathbf{p}\right) = \mathbf{f}\left(\begin{pmatrix} 0.985 \\ -0.29775 \end{pmatrix}, 0.1, \mathbf{p}\right) = \begin{pmatrix} -0.29775 \\ -2.955 \end{pmatrix}.
 \end{aligned}$$

Por lo tanto,

$$\begin{aligned}
 \mathbf{x}_1 &= \mathbf{x}_0 + \frac{0.1}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \\
 &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \frac{0.1}{6} \begin{pmatrix} -0.89775 \\ -17.91 \end{pmatrix} = \boxed{\begin{pmatrix} 0.9850375 \\ -0.2985 \end{pmatrix}}.
 \end{aligned}$$

4.2.2. Forma matricial de la iteración para este oscilador

El sistema puede escribirse como

$$\mathbf{x}' = A\mathbf{x}, \quad A = \begin{pmatrix} 0 & 1 \\ -\alpha & 0 \end{pmatrix}.$$

Para una ecuación lineal autónoma, un paso de RK4 equivale a aplicar el polinomio de Taylor de grado 4 de la exponencial matricial:

$$\mathbf{x}_{n+1} = \left(I + hA + \frac{h^2 A^2}{2} + \frac{h^3 A^3}{6} + \frac{h^4 A^4}{24} \right) \mathbf{x}_n. \quad (41)$$

Como $A^2 = -\alpha I$, resulta

$$\mathbf{x}_{n+1} = \left[\left(1 - \frac{\alpha h^2}{2} + \frac{\alpha^2 h^4}{24} \right) I + \left(h - \frac{\alpha h^3}{6} \right) A \right] \mathbf{x}_n. \quad (42)$$

Para $\alpha = 3$ y $h = 0.1$,

$$\boxed{\begin{pmatrix} x_{1,n+1} \\ x_{2,n+1} \end{pmatrix} = \begin{pmatrix} 0.9850375 & 0.0995 \\ -0.2985 & 0.9850375 \end{pmatrix} \begin{pmatrix} x_{1,n} \\ x_{2,n} \end{pmatrix}}. \quad (43)$$

Esta forma es particular del oscilador lineal y sirve para verificar algebraicamente los valores producidos por la implementación general de RK4.

4.2.3. Condición inicial y primeros 10 pasos de RK4

La fila $n = 0$ corresponde a la condición inicial; las filas 1 a 10 se obtienen aplicando sucesivamente la ecuación (43).

Cuadro 2: Primeras diez iteraciones de RK4 para $\alpha = 3$ y $h = 0.1$.

n	t_n	$x_{1,n}$	$x_{2,n}$
0	0.0	1.0000000000	0.0000000000
1	0.1	0.9850375000	-0.2985000000
2	0.2	0.9405981264	-0.5880673875
3	0.3	0.8680117219	-0.8600369699
4	0.4	0.7694504180	-1.1062701658
5	0.5	0.6478636346	-1.3193985482
6	0.6	0.5068898194	-1.4930443423
7	0.7	0.3507475685	-1.6220112775
8	0.8	0.1841093858	-1.7024400829
9	0.9	0.0119618609	-1.7319239748
10	1.0	-0.1605435539	-1.7095806779

Obsérvese, por ejemplo, que el valor de la primera fila calculada coincide con el desarrollo explícito de $k_1, \dots, k_4$ realizado arriba.

4.2.4. Código para repetir el Ejercicio 6 usando RK4

```
import numpy as np
import matplotlib.pyplot as plt

def euler(f, x, t, h, p):
    return x + h * f(x, t, p)

def rk4(f, x, t, h, p):
    """Computa un paso de integración con Runge-Kutta de orden 4."""
    k1 = f(x, t, p)
    k2 = f(x + 0.5 * h * k1, t + 0.5 * h, p)
    k3 = f(x + 0.5 * h * k2, t + 0.5 * h, p)
    k4 = f(x + h * k3, t + h, p)

    x_siguiente = x + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
    return x_siguiente

def integrador_ode(m, f, x0, t0, tf, npasos, p):
    h = (tf - t0) / npasos

    t_puntos = np.zeros(npasos + 1)
    x_puntos = np.zeros((npasos + 1, len(x0)))
    t_puntos[0] = t0
    x_puntos[0] = x0

    for i in range(npasos):
        x_actual = x_puntos[i]
        t_actual = t_puntos[i]

        if m == 'e':
            x_siguiente = euler(f, x_actual, t_actual, h, p)
        elif m == 'rk4':
            x_siguiente = rk4(f, x_actual, t_actual, h, p)
        else:
            raise ValueError("Método no soportado. Use 'e' o 'rk4'.")

        t_puntos[i+1] = t_puntos[i] + h
        x_puntos[i+1] = x_siguiente
```

```

        x_puntos[i+1] = x_siguiete
        t_puntos[i+1] = t_actual + h

    return t_puntos, x_puntos

def oscilador_armonico(x, t, p):
    alpha = p[0]
    x1 = x[0]
    x2 = x[1]
    return np.array([x2, -alpha * x1])

x_inicial = np.array([1.0, 0.0])
tiempo_inicial = 0.0
tiempo_final = 10.0
h_step = 0.1
n_pasos = int((tiempo_final - tiempo_inicial) / h_step)
parametros_alpha = np.array([3.0])

# Integración con RK4
tiempos_rk4, solucion_rk4 = integrador_ode(
    'rk4', oscilador_armonico, x_inicial,
    tiempo_inicial, tiempo_final, n_pasos, parametros_alpha
)

x1_rk4 = solucion_rk4[:, 0]
x2_rk4 = solucion_rk4[:, 1]

# Solución exacta para comparar
alpha = parametros_alpha[0]
omega = np.sqrt(alpha)
tiempos_exactos = np.linspace(tiempo_inicial, tiempo_final, 1000)
x1_exacta = np.cos(omega * tiempos_exactos)
x2_exacta = -omega * np.sin(omega * tiempos_exactos)

# x1(t) y x2(t) frente a t: solución exacta y RK4
plt.figure(figsize=(9, 5))
plt.plot(tiempos_exactos, x1_exacta, color='C0', label=r'$x_1$ exacta')
plt.scatter(tiempos_rk4, x1_rk4, s=14, marker='s', color='C0', label=r'$x_1$ RK4')
plt.plot(tiempos_exactos, x2_exacta, color='C1', label=r'$x_2$ exacta')
plt.scatter(tiempos_rk4, x2_rk4, s=14, marker='s', color='C1', label=r'$x_2$ RK4')
plt.xlabel(r'Tiempo $t$')
plt.ylabel(r'$x_1(t)$, $x_2(t)$')
plt.legend()
plt.grid(True)
plt.show()

# Diagrama de fases: x2(t) frente a x1(t)
plt.figure(figsize=(6, 6))
plt.plot(x1_exacta, x2_exacta, color='C0', label='Exacta')
plt.scatter(x1_rk4, x2_rk4, s=14, marker='s', color='C0', label='RK4')
plt.xlabel(r'$x_1(t)$')
plt.ylabel(r'$x_2(t)$')
plt.legend()
plt.grid(True)
plt.show()

```

Listing 6: Oscilador armónico integrado con RK4

4.2.5. Resultado gráfico esperado con RK4

En la primera figura se muestran los estados $x_1(t)$ y $x_2(t)$ obtenidos con RK4 junto con la solución exacta

$$x_1(t) = \cos(\sqrt{3}t), \quad x_2(t) = -\sqrt{3}\sin(\sqrt{3}t).$$

Las líneas continuas representan la solución exacta y los marcadores cuadrados los valores calculados con RK4. Para $h = 0.1$, ambos quedan prácticamente superpuestos. La segunda figura muestra la misma comparación en el plano de fases.

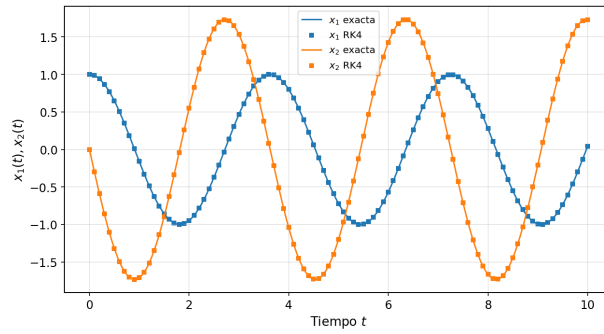


Figura 4: Solución del oscilador armónico con RK4 y $h = 0.1$, comparada con la solución exacta.

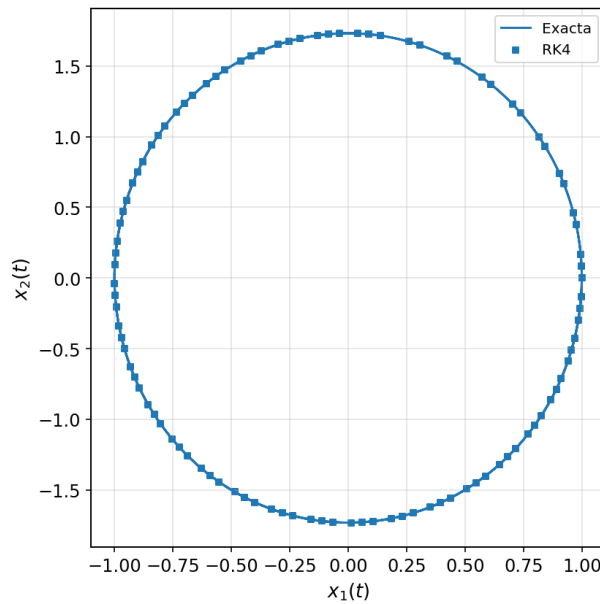


Figura 5: Diagrama de fases $x_2(t)$ frente a $x_1(t)$: RK4 comparado con la solución exacta.

4.3. Comparación entre Euler y RK4

Para comparar ambos métodos de manera directa, integramos el mismo problema con el mismo paso $h = 0.1$ y las mismas 100 iteraciones. La solución exacta en $t = 10$ es

$$\mathbf{x}(10) = \begin{pmatrix} 0.0417363545 \\ 1.7305416002 \end{pmatrix}. \quad (44)$$

Los resultados numéricos al final del intervalo son:

Cuadro 3: Comparación al tiempo final $t = 10$ para $h = 0.1$.

Método	$x_1(10)$	$x_2(10)$	$\ \mathbf{x}_{num} - \mathbf{x}_{exact}\ _2$	$E(10)$
Exacta	0.0417363545	1.7305416002	0	1.5000000000
Euler	-0.5613663479	7.5306373842	5.8313672473	28.8279479713
RK4	0.0416071747	1.7305185508	0.0001312206	1.4999439620

donde usamos como energía del oscilador

$$E(t) = \frac{1}{2}x_2(t)^2 + \frac{\alpha}{2}x_1(t)^2. \quad (45)$$

Para la solución exacta, $E(t) = E(0) = \alpha/2 = 1.5$ para todo t .

La diferencia es muy marcada. Euler es un método de primer orden: con este paso introduce un error apreciable en fase y amplitud, y además aumenta artificialmente la energía. Como se vio en el Ejercicio 6, su matriz de un paso tiene $\det(A_E) = 1.03 > 1$, por lo que la trayectoria numérica se abre hacia afuera.

RK4, en cambio, es de cuarto orden y para el mismo $h = 0.1$ sigue de cerca tanto la fase como la amplitud de la solución exacta. Para este problema, la matriz de un paso de RK4 de la ecuación (43) tiene

$$\det(A_{RK4}) \approx 0.9999996264, \quad (46)$$

muy cercano a uno. Por ello el cambio artificial de amplitud en 100 pasos es muy pequeño. Es importante notar que RK4 *no* conserva exactamente la energía del oscilador: en este caso produce una disminución minúscula, mientras que Euler explícito produce un crecimiento muy grande.

La comparación de los diagramas de fases hace visible esta diferencia:

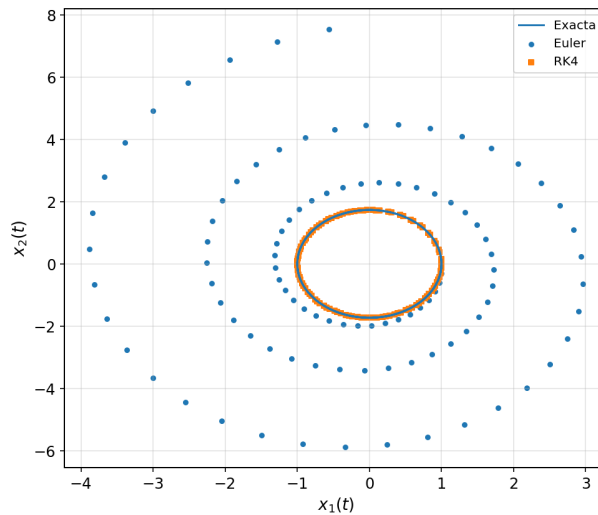


Figura 6: Comparación del diagrama de fase $x_2(t)$ frente a $x_1(t)$: solución exacta, Euler y RK4 para $h = 0.1$.

Observaciones. Se observa una diferencia clara: con el mismo paso de integración, RK4 reproduce el oscilador armónico con una precisión muy superior a Euler. Euler desarrolla una espiral creciente por acumulación de error numérico, mientras que RK4 mantiene una órbita prácticamente cerrada durante el intervalo considerado.